

C++ and Object Oriented Programming



Unit – 5 (Topic – 2) Exception handling & Introduction To Template STL

**Prepared By
Prof. Shah Brijesh**

Exception Handling

Q : Write short note on exception handling:

- > Exception handling means error handling.
- > When we create programs, it might have bugs.
- > The two most common types of bugs are logic errors and syntactic errors.
- > The logic errors occurs due to poor understanding of th problem and solution procedure and syntactic errors arise due to poor understanding of the language itself.
- > We can detect these errors by using debugging and resting procedures.
- > But some times, we may found that program returns errors other than logic and syntax errors.
- > They are known as **exceptions**.
- > Exceptions are unusual conditions that a program may encounter while executing.
- > It might include division by zero, access to an array outside of its bounds or running out of memory or disk space.
- > When a program encounters an exceptional condition, it is important that it is identified and dealt with effectively.
- > ANSI C++ provides built-in language features to detect and handle exceptions which are basically run time errors.
- > Exception handling was not part of the original c++.

Exception Handling

Q : Write short note on exception handling:

Basics Of Exception Handling:

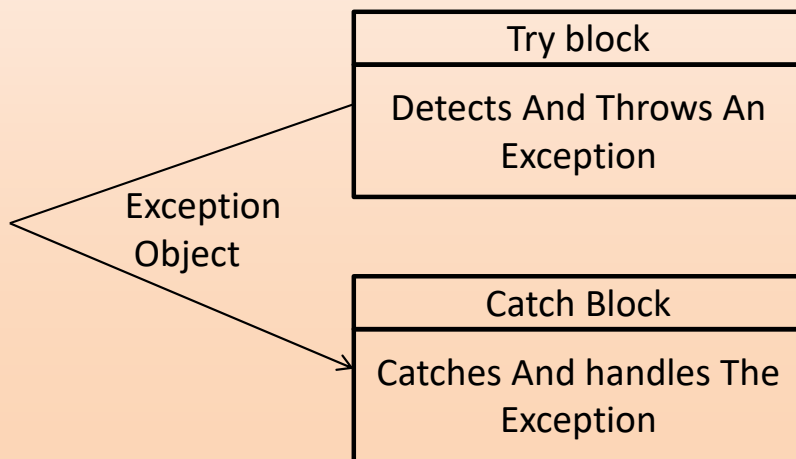
- > Exceptions are of two kinds.
- > Synchronous exceptions and asynchronous exceptions.
- > Errors such as “Out of range index” and “over flow” belong to the synchronous type exceptions.
- > The error that are caused by events beyond the control of the program (such as keyboard interrupts) are called asynchronous exceptions.
- > The proposed exception handling mechanism in C++ designed to handle only synchronous exceptions.
- > The mechanism suggests a separate error handling code that performs the following tasks.
 1. Find The Problem (Hit The Exception)
 2. Inform That An Error Has Occurred (Throw The Exception)
 3. Receive The Error Information (Catch The Information)
 4. Take Corrective Actions (handle The Exception).

Exception Handling

Q : Write short note on exception handling:

Exception Handling Mechanism:

- > C++ Exception handling mechanism is basically built upon three keywords.
- > Namely try, throw and catch.
- > The keyword try is used to preface a block of statements which may generate exceptions.
- > This block of statements is known as try block.
- > When an exception is detected, it is thrown using a throw statement in the try block.
- > A catch block defined by the keyword catch 'catches' the exception 'thrown' by the throw statement in the try block, and handles it appropriately.



Exception Handling

Q : Write short note on exception handling:

Exception Handling Mechanism:

-> The catch block catches an exception must immediately follow the try block that throws the exception.

-> The general form of these two blocks are as follows.

```
.....  
.....  
try  
{  
    .....  
    .....  
    throw exception;  
    .....  
    .....  
}  
catch(try arg)  
{  
    .....  
    .....  
    .....  
    .....  
}  
.....  
.....
```

Exception Handling

Q : Write short note on exception handling:

Exception Handling Mechanism:

- > When the try block throws an exception, the program control leaves the try block and enters the catch statement of the catch block.
- > Note that exceptions are objects used to transmit information about a problem.
- > If the type of object thrown matches the arg type in the catch statement, then catch block is executed for handling the exception.
- > If they do not match, the program is aborted with the help of the abort() function which is invoked by default.
- > When no exception is detected and thrown, the control goes to the statement immediately after the catch block.
- > That is, the catch block is skipped.

```
#include<iostream.h>
```

```
void mai()
```

```
{
```

```
    int a,b;
```

```
    cout<<"Enter values of a and b:";
```

```
    cin>>a>>b;
```

```
    int x = a-b;
```

Exception Handling

Q : Write short note on exception handling:

Exception Handling Mechanism:

```
try
{
    if(x!=0)
    {
        cout<<"\n Result ="<<a/x;
    }
    else
    {
        throw(x);
    }
}
catch(int i)
{
    cout<<"Exception caught : x ="<<x;
}
cout<<"\n End";
}
```

Templates

Q : What is Template? Explain function template and class template with example:

- > Templates are the foundation of generic programming, which involves writing code in a way that is independent of any particular type.
- > A template is a blueprint or formula for creating a generic class or a function.
- > The library containers like iterators and algorithms are examples of generic programming and have been developed using template concept.
- > There is a single definition of each container, such as **vector**, but we can define many different kinds of vectors for example, **vector <int>** or **vector <string>**.
- > You can use templates to define functions as well as classes, let us see how they work.

Function Template

- > The general form of a template function definition is shown here –

```
template <class type>
ret-type func-name(parameter list)
{
    // body of function
}
```

- > Here, type is a placeholder name for a data type used by the function. This name can be used within the function definition.

Templates

Q : What is Template? Explain function template and class template with example:

Function Template

-> The following is the example of a function template that returns the maximum of two values –

```
template <typename T>
inline T const& Max (T const& a, T const& b) {
    return a < b ? b:a;
}

void main () {
    int i = 39;
    int j = 20;
    cout << "Max(i, j): " << Max(i, j) << endl;
    double f1 = 13.5;
    double f2 = 20.7;
    cout << "Max(f1, f2): " << Max(f1, f2) << endl;
    string s1 = "Hello";
    string s2 = "World";
    cout << "Max(s1, s2): " << Max(s1, s2) << endl;
}
```

Templates

Q : What is Template? Explain function template and class template with example:

Class Template:

-> Just as we can define function templates, we can also define class templates.

-> The general form of a generic class declaration is shown here –

```
template <class type>
class class-name
{
    .....
};
```

-> Here, **type** is the placeholder type name, which will be specified when a class is instantiated.

-> We can define more than one generic data type by using a comma-separated list.

-> A class created from a class template is called template class.

-> The syntax for defining an object of a template class is:

```
class-name <type> objectname (arglist);
```

-> This process of creating a specific class from a class template called instantiation.

-> The compiler will perform the error analysis only when an instantiation takes place.

Templates

Q : What is Template? Explain function template and class template with example:

Class Template:

-> For Example

template <class T>

class Vector

{

 T *v;

 public:

 Vector ()

 {

 }

 Vector(T *a)

 {

 for(int i=0; i<3 ; i++)

 v[i] = a[i];

 }

Templates

Q : What is Template? Explain function template and class template with example:

Class Template:

```
        T operator*(Vector &y)
        {
            T sum = 0;
            for(int i=0; i<3 ; i++)
                sum += this->v[i] * y.v[i];
            return sum;
        }
};

void main()
{
    int x[3] = {1, 2, 3}, y[3] = {4, 5 ,6};
    Vector <int> v1;
    Vector <int> v2;
    v1 = x;
    v2 = y;
    int R = v1 * v2;
    cout<<"\n R = "<<R;        }
```

Standard Template Library [STL]

Q : Write short note on STL [Standard Template Library]

STL:

-> The Standard Template Library (STL) is a set of C++ template classes to provide common programming data structures and functions such as lists, stacks, arrays, etc.

-> It is a library of container classes, algorithms and iterators.

-> It is a generalized library and so, its components are parameterized.

STL has four components

1. Algorithms
2. Containers
3. Functions
4. Iterators

Standard Template Library [STL]

Q : Write short note on STL [Standard Template Library]

1. Algorithms

-> The header algorithm defines a collection of functions especially designed to be used on ranges of elements.

-> They act on containers and provide means for various operations for the contents of the containers.

-> Algorithm

- Sorting
- Searching
- Important STL Algorithms
- Useful Array Algorithms
- Partition Operations

-> Numeric

- valarray class

Standard Template Library [STL]

Q : Write short note on STL [Standard Template Library]

2. Containers

-> Containers or container classes store objects and data. There are in total seven standard “first-class” container classes and three container adaptor classes and only seven header files that provide access to these containers or container adaptors.

-> Sequence Containers: implement data structures which can be accessed in a sequential manner.

- [vector](#)
- [list](#)
- [deque](#)
- [arrays](#)
- [forward list](#)(Introduced in C++11)

-> Container Adaptors : provide a different interface for sequential containers.

- [queue](#)
- [priority queue](#)
- [stack](#)

Standard Template Library [STL]

Q : Write short note on STL [Standard Template Library]

-> Associative Containers : implement sorted data structures that can be quickly searched ($O(\log n)$ complexity).

- [set](#)
- [multiset](#)
- [map](#)
- [Multimap](#)

3. Functions

-> The STL includes classes that overload the function call operator. Instances of such classes are called function objects or functors.

-> Functors allow the working of the associated function to be customized with the help of parameters to be passed.

[Functors](#)

4. Iterators

-> As the name suggests, iterators are used for working upon a sequence of values. They are the major feature that allow generality in STL.

[Iterators](#)

Standard Template Library [STL]

Iterators in C++ STL

-> Iterators are used to point at the memory addresses of [STL](#) containers. They are primarily used in sequence of numbers, characters etc. They reduce the complexity and execution time of program.

Operations of iterators :-

1. begin() :- This function is used to return the **beginning position** of the container.

2. end() :- This function is used to return the **end position** of the container.

// C++ code to demonstrate the working of

// iterator, begin() and end()

```
#include<iostream>
```

```
#include<iterator> // for iterators
```

```
#include<vector> // for vectors
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    vector<int> ar = { 1, 2, 3, 4, 5 };
```

```
    // Declaring iterator to a vector
```

```
    vector<int>::iterator ptr;
```

```
    // Displaying vector elements using begin() and end()
```

```
    cout << "The vector elements are : ";
```

```
    for (ptr = ar.begin(); ptr < ar.end(); ptr++)
```

```
        cout << *ptr << " ";
```

```
    return 0;
```

```
}
```

Standard Template Library [STL]

Iterators in C++ STL

3. advance() :- This function is used to **increment the iterator position** till the specified number mentioned in its arguments.

```
// C++ code to demonstrate the working of
// advance()
#include<iostream>
#include<iterator> // for iterators
#include<vector> // for vectors
using namespace std;
int main()
{
    vector<int> ar = { 1, 2, 3, 4, 5 };
    // Declaring iterator to a vector
    vector<int>::iterator ptr = ar.begin();
    // Using advance() to increment iterator position
    // points to 4
    advance(ptr, 3);
    // Displaying iterator position
    cout << "The position of iterator after advancing is : ";
    cout << *ptr << " ";
    return 0;
}
```

Standard Template Library [STL]

Iterators in C++ STL

4. next() :- This function **returns the new iterator** that the iterator would point after **advancing the positions** mentioned in its arguments.

5. prev() :- This function **returns the new iterator** that the iterator would point **after decrementing the positions** mentioned in its arguments.

```
// C++ code to demonstrate the working of
// next() and prev()
#include<iostream>
#include<iterator> // for iterators
#include<vector> // for vectors
using namespace std;
int main()
{
    vector<int> ar = { 1, 2, 3, 4, 5 };
    // Declaring iterators to a vector
    vector<int>::iterator ptr = ar.begin();
    vector<int>::iterator ftr = ar.end();
    // Using next() to return new iterator points to 4
    auto it = next(ptr, 3);
    // Displaying iterator position
    cout << "The position of new iterator using next() is : ";
    cout << *it << " ";
    cout << endl;
    // Displaying iterator position
    cout << "The position of new iterator using prev() is : ";
    cout << *it1 << " ";
    cout << endl;
    return 0;
}
```

Standard Template Library [STL]

Iterators in C++ STL

6. inserter() :- This function is used to **insert the elements at any position** in the container. It accepts **2 arguments, the container and iterator to position where the elements have to be inserted**.

// C++ code to demonstrate the working of inserter()

```
#include<iostream>
```

```
#include<iterator> // for iterators
```

```
#include<vector> // for vectors
```

```
using namespace std;
```

```
int main()
```

```
{    vector<int> ar = { 1, 2, 3, 4, 5 };
```

```
    vector<int> ar1 = {10, 20, 30};
```

```
    // Declaring iterator to a vector
```

```
    vector<int>::iterator ptr = ar.begin();
```

```
    // Using advance to set position
```

```
    advance(ptr, 3);
```

```
    // copying 1 vector elements in other using inserter() inserts ar1 after 3rd position in ar
```

```
    copy(ar1.begin(), ar1.end(), inserter(ar,ptr));
```

```
    // Displaying new vector elements
```

```
    cout << "The new vector after inserting elements is : ";
```

```
    for (int &x : ar)
```

```
        cout << x << " ";
```

```
    return 0;
```

```
}
```